ChatGPT said:

Meeting Rooms 1, 2 & LC 2406 — Interview Prep

For Infosys preparation, learn these three problems in this order. They look similar, but each teaches a different greedy technique.

- **LC 252 — Meeting Rooms:** Can one person attend every meeting?
- **LC 253 — Meeting Rooms II:** What is the minimum number of rooms required?
- **LC 2406 — Divide Intervals Into Minimum Number of Groups:** What is the minimum number of groups needed when overlapping intervals cannot share a group?

I'll use **Java** for both an easier solution and the optimal solution, with examples and test cases.

1. LC 252 — Meeting Rooms

Difficulty: Easy · Main concept: Sorting + intervals

Problem: Given an array `intervals`, where `intervals[i] = [start, end]`, return `true` if a person can attend all meetings without any overlap. Otherwise, return `false`.

Test cases

Input	Output	Reason
`[[0,30],[5,10],[15,20]]`	`false`	Meeting `[5,10]` overlaps `[0,30]`
`[[7,10],[2,4]]`	`true`	No overlap
`[[1,5],[5,8]]`	`true`	One meeting ends as the next begins
`[[1,4],[2,3],[5,7]]`	`false`	First two overlap
`[[1,2]]`	`true`	Only one meeting

Easier solution — Compare every pair

Check every pair of meetings for overlap.

Two intervals overlap when:

$$a.start < b.end \quad \text{and} \quad b.start < a.end$$

java

```
class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        int n = intervals.length;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (intervals[i][0] < intervals[j][1]
                    && intervals[j][0] < intervals[i][1]) {
                    return false;
                }
            }
        }

        return true;
    }
}
```

- Time: $O(n^2)$
- Space: $O(1)$, excluding input.

Optimal solution — Sort by start time

Once the meetings are sorted, you only need to compare each meeting with the previous one.

java

```
import java.util.Arrays;

class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals,
            (a, b) -> Integer.compare(a[0], b[0]));

        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] < intervals[i - 1][1]) {
                return false;
            }
        }
```

```
        return true;
    }
}
```

- Time: $O(n \log n)$
- Extra space: $O(1)$ auxiliary space for the scan, excluding sorting implementation details.

Remember: For LC 252, sorting by start time and checking for overlap is enough. You don't need a heap.

Official problem: LeetCode 252 — Meeting Rooms ↗

2. LC 253 — Meeting Rooms II

Difficulty: Medium · Main concept: Greedy + Min-Heap

Problem: Given meeting intervals, return the **minimum number of conference rooms required**. LeetCode Wiki +1

Test cases

Input	Output
`[[0,30],[5,10],[15,20]]`	2
`[[7,10],[2,4]]`	1
`[[1,5],[2,6],[3,7]]`	3
`[[1,4],[4,8]]`	1
`[[1,10],[2,3],[4,5],[6,7]]`	2
`[[1,2]]`	1

Easier solution — Compare every meeting pair

Instead of assigning rooms individually, count how many meetings overlap each other, then find the maximum number of simultaneous meetings.

For learning purposes, a simpler brute-force approach is to check every meeting's start time and count the meetings active at that time.

java

```java
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        int maxRooms = 0;

        for (int[] meeting : intervals) {
            int time = meeting[0];
            int active = 0;

            for (int[] other : intervals) {
                if (other[0] <= time && time < other[1]) {
                    active++;
                }
            }

            maxRooms = Math.max(maxRooms, active);
        }

        return maxRooms;
    }
}
```

- Time: $O(n^2)$
- Extra space: $O(1)$

Why does this work? The number of rooms required equals the maximum number of meetings happening simultaneously.

Optimal solution — Min-Heap

Use a min-heap to track the ending time of each room's current meeting.

java

```java
import java.util.*;

class Solution {
    public int minMeetingRooms(int[][] intervals) {
```

```
        if (intervals.length == 0) return 0;

        Arrays.sort(intervals,
            (a, b) -> Integer.compare(a[0], b[0]));

        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        for (int[] meeting : intervals) {
            int start = meeting[0];
            int end = meeting[1];

            // Reuse a room if its meeting has ended
            if (!minHeap.isEmpty() &&
                minHeap.peek() <= start) {
                minHeap.poll();
            }

            // Assign the current meeting to a room
            minHeap.offer(end);
        }

        return minHeap.size();
    }
}
```

- Time: $O(n \log n)$
- Space: $O(n)$

Dry run

Input: `[[0,30],[5,10],[15,20]]`

Meeting	Action	Heap (end times)
`[0,30]`	Create room 1	`[30]`
`[5,10]`	Create room 2	`[10,30]`
`[15,20]`	Reuse room ending at 10	`[20,30]`

Final answer: `2` rooms.

Why a min-heap? We need to know which room becomes free earliest. The smallest end time is at the top.

Official problem: LeetCode 253 — Meeting Rooms II ↗

3. LC 2406 — Divide Intervals Into Minimum Number of Groups

Difficulty: Medium · Main concept: Greedy + Min-Heap

Problem: Given intervals `[left, right]`, divide them into the minimum number of groups such that **no two intervals in the same group overlap**. LeetCode +1

Important difference: LC 2406 uses **inclusive intervals**. Therefore, `[1,5]` and `[5,8]` overlap because both contain `5`.

Test cases

Input	Output	Reason
`[[5,10],[6,8],[1,5],[2,3],[1,10]]`	3	Maximum overlap is 3
`[[1,3],[5,6],[8,10],[11,13]]`	1	No intervals overlap
`[[1,5],[5,8]]`	2	Both contain 5
`[[1,2],[2,3],[3,4]]`	2	Adjacent intervals overlap at endpoints
`[[1,10],[2,9],[3,8],[4,7]]`	4	All four overlap
`[[1,1]]`	1	One interval

Easier solution — Count overlaps directly

For every interval's left endpoint, count how many intervals contain that point. The largest count is the answer.

Why check left endpoints? The maximum overlap must occur at the start of at least one interval.

java

```
class Solution {
    public int minGroups(int[][] intervals) {
        int maxGroups = 0;

        for (int[] current : intervals) {
            int point = current[0];
            int count = 0;

            for (int[] interval : intervals) {
                if (interval[0] <= point &&
                    point <= interval[1]) {
                    count++;
                }
            }

            maxGroups = Math.max(maxGroups, count);
        }

        return maxGroups;
    }
}
```

- Time: $O(n^2)$
- Extra space: $O(1)$

This is a good way to understand the logic before learning the heap.

Optimal solution — Sort + Min-Heap

The heap stores the ending point of each group's last interval.

1. Sort intervals by their left endpoint.
2. If the smallest ending point is **strictly less than** the current left endpoint, reuse that group.
3. Otherwise, create another group.
4. The final heap size is the minimum number of groups.

java

```
import java.util.*;

class Solution {
    public int minGroups(int[][] intervals) {
        Arrays.sort(intervals,
            (a, b) -> Integer.compare(a[0], b[0]));

        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        for (int[] interval : intervals) {
            int left = interval[0];
            int right = interval[1];

            // Strictly less because intervals are inclusive
            if (!minHeap.isEmpty() &&
                minHeap.peek() < left) {
                minHeap.poll();
            }

            minHeap.offer(right);
        }

        return minHeap.size();
    }
}
```

- Time: $O(n \log n)$
- Space: $O(n)$

Dry run

Input: `[[1,5],[2,3],[5,8]]`

Interval	Action	Heap
`[1,5]`	Create group 1	`[5]`
`[2,3]`	Create group 2	`[3,5]`

`[5,8]`	Reuse group ending at 3	`[5,8]`

Final answer: `2` groups.

Notice that `[5,8]` cannot reuse the group ending at `5`, because these are inclusive intervals. The group ending at `3` is reusable.

Official problem: LeetCode 2406 — Divide Intervals Into Minimum Number of Groups ↗

Final revision sheet for Infosys

Problem	Main technique	Key condition
LC 252	Sorting	`currentStart < previousEnd` means overlap
LC 253	Sorting + Min-Heap	Reuse room if `earliestEnd <= start`
LC 2406	Sorting + Min-Heap	Reuse group if `earliestEnd < left`

How to remember which greedy condition to use

- **LC 252:** Just detect overlap.
- **LC 253:** Meetings ending at time `t` can free a room for a meeting starting at `t`.
- **LC 2406:** Intervals sharing even one endpoint overlap, so the previous interval must end strictly before the next begins.

My recommendation: First understand LC 252, then solve LC 253, and finally solve LC 2406. The last two use almost the same heap logic; the main difference is the equality condition.

You said:

out of these 3 which is n meetings in one room

ChatGPT said:

N Meetings in One Room is a different problem from these three.

- **GFG:** N Meetings in One Room ↗
- **LeetCode equivalent:** LC 435 — Non-overlapping Intervals ↗ is closely related, but asks for the minimum number of intervals to remove.